Pintos Environment Setup

CSE 421/521

Spring 2026

1 Introduction

This document has a lot of information in it, we recommend you read it more than once *before* actually doing anything.
Order of operations is very important for setting up your environment, and the order of sections in this document is intentional.

2 Docker Installation

We will be using Docker Desktop as the host platform for Pintos development in this course, and this is the only environment *we are supporting*.

Docker will prompt you several times to login to an account, you ***do not*** need a docker account to use Docker Desktop, or any of its features for this course.

2.1 Linux

Note that the Docker engine, and Docker Desktop ***are not the same***, and we are supporting Docker Desktop.
You should go through their documentation, *read through it first*, and then follow the instructions for your specific distribution.

2.2 macOS

Docker Desktop supports both Apple Silicon(arm64) and x86_64 based Macs, *read through the instructions first*, and then follow them for your specific machine architecture. The instructions are here.
There are permissions considerations for installing Docker Desktop on macOS, you should also read through that information here.

2.2.1 Command line utilities

When you run docker commands from the terminal application, you may encounter macOS asking you to install command line utilities, *you will need to install them when prompted*.

2.2.2 Warning on Apple Silicon

If you receive a warning like:

```
WARNING: The requested image's platform (linux/amd64) does not match the detected host
    platform (linux/arm64/v8) and no specific platform was requested
```

you can safely ignore it, as the original image was for `amd64`, and it is just informing you of that.

2.3 Windows

Setup for Windows is a little more complicated than other platforms.

Windows Subsystem for Linux 2 (WSL2) is the most straightforward path, and the supported one for our course. Setup instructions for that can be found here.
The setup and install instructions for Docker Desktop on Windows can be found here.

There are permissions considerations for Docker Desktop on Windows, you should read through that information here.

3 Using Docker

3.1 Building the Docker image

The `Dockerfile` can be found on Piazza under the Resources tab. It would be wise to move that file to a location specifically for this project, however you organize your data.
The directory where the `Dockerfile` lives is where the following setup commands will be run from.
In general, if you are unsure about a command you are running, you can find documentation using the `man` pages, i.e. `man docker-build` or `man docker-run`, etc.

A Docker image is the standard for which Docker containers are built from. You should only have to build this image once, and use it to create containers to develop within.
While being in *the same directory* as the `Dockerfile`, you can build the image with the following command in a terminal:

```
docker build . -t pintos_image
```

You could replace `pintos_image` with any appropriate name for how you want to organize it.
The flag `-t` flag, in this instance refers to tagging the image with the name you provided.

3.2 Creating a container

After creating the image, you can now use it to create a container for developing within.
In a terminal, you can run:

```
docker run --name pintos_container -it pintos_image
```

In this command, we are naming the container `pintos_container`. The flags `-it` tell docker to run the container with an interactive terminal.
`pintos_image` being appended after the flags tells docker which image we want to create the container from.
After running that, you should shortly see something resembling a Linux command prompt.

If you have created a container before(and have not deleted it), you can run it with the following command:

```
docker start -i pintos_container
```

`pintos_container` can be replaced with whatever you named the container you previously created.
You can also stop a started container using the following:

```
docker stop pintos_container
```

`pintos\_container` can again be replaced with whatever you named the container you previously created.

If you want to list currently existing containers (started or not), you can run:

```
docker ps -a
```

In this case, `-a` tells docker to list both running and stopped containers.

4 Developing with Docker

With the container now created, you can now start development on your project!

The pintos source lives in `/home/pintos`, and when you enter the container, you will be dropped at `/`

Any files we talk about already exist, unless we otherwise state it!

You can navigate to the pintos source using the change directory command:

```
cd /home/pintos
```

4.1 Git configuration within the container

To be able to `push` and `pull` from git within the container, you will need to setup an `ssh` key *inside the container*. ***This is separate from other keys you may have created in the past.***

From inside the container you can run the following to generate a new `ssh` key:

```
ssh-keygen
```

following the on screen instructions, and then copying the newly created public key to your GitHub account. GitHub has more in-depth documentation for this process here.

4.2 Editing code *outside* the container

With GitHub setup, you can now edit files from the container in a couple of ways.

You can use Docker to copy files to and from the container and your local machine:

```
docker cp <source> <destination>
```

For example, to copy a directory from your local machine to the container, you could do the following:

```
docker cp src/ pintos_container:dest/
```

This would copy the `src/` directory into the `dest/` directory within the container, assuming the name of the container is `pintos_container`
Copying files and directories out of the container works the reverse of the previous example.

```
docker cp pintos_container:src/ dest/
```

This would copy the `src/` directory from the container into the `dest/` directory on your local machine.

4.3 Editor integration

4.3.1 Emacs

For Emacs versions 29 or greater, there is built in functionality for this, information can be found here.
For versions 28 and earlier, there exists a method to achieve similar functionality, that can be found here.

4.3.2 VSCode

There also exist extensions on VSCode for Docker and Containers, they are available here and here.

4.4 Editing code *within* the container

You can also edit code directly within the container by installing your programmers editor of choice, i.e. `emacs`, `vim`, etc. using `apt`:

```
apt install vim
```

for instance, that would install `vim` into the container, which you could then run on the command line

5 Debugging with `gdb`

It can be incredibly helpful to be able step through the source of pintos while it is running, 'gdb' can help us do this, but getting setup is *a little different* from how you might be used to. `gdb` still requires an executable/object to debug, which for our purposes would be `kernel.o`.

`pintos` has a command-line flag for using `gdb`, which is `--gdb`
So, if we wanted use this flag for running debugging the `alarm-multiple` test, we would first change directories to where `alarm-multiple` is:

```
cd pintos/src/threads/build
```

and then we can run pintos with debugging enabled:

```
pintos --gdb -- run alarm-multiple
```

after this starts, we should see at around the last line of output:

```
Waiting for gdb connection on localhost:1234
```

And now, ***in a new terminal window***, change directories to where `kernel.o` is, which for this test is:

```
cd pintos/src/threads/build
```

and now we can start `gdb` on `kernel.o`:

```
gdb kernel.o
```

and inside the `gdb` prompt, we can now attach to the remote debugging session:

```
target remote localhost:1234
```

and now `gdb` should be attached to the running pintos kernel!

you can now do all the usual things you can do in 'gdb', like setting `breakpoints`, `continue`ing the execution of the kernel, `print`ing values, etc.